

s2.05 — Multi-tenant authorization policy service for certificate and key operations

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Compile, sign, and atomic generation switch — IC3 baseline](#)
- [Decision evaluation, caching, and freshness — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: JD-fit preparation addition in [the bank](#)
 - No first-hand report of this exact prompt.
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce OCI IAM internals or an interviewer's accepted answer.
- Why this is hard
 - Naive: `if role == admin` inside each service, with a roles table edited by hand.
 - First break: a removed operator keeps signing for minutes because every service cached "admin", and nobody can say which policy version allowed a key deletion.
 - Second break: the policy service goes down and every bundle read and HSM signing call fails with it
 - The authorization control plane becomes the data plane's single point of failure.
 - Minimum primitives
 - Immutable policy versions
 - Bindings compiled into a signed per-tenant generation
 - Default deny
 - Server-resolved attributes
 - A cache key that includes the generation
 - A freshness rule for operations that destroy keys or trust.
- 🗯️ "Key authorization is a freshness problem wearing a caching costume: cache a signed generation for stability, and refuse to sign or delete on a stale one."
- Opening script
 - "I will treat this as a policy control plane that publishes signed, immutable per-tenant generations to enforcement agents inside the certificate and key data planes."
 - "First I confirm who authors policy, which operations are sensitive, and how fast a revoked binding must stop working."

- “Then I size decision rate and generation size, which forces evaluation to be agent-local rather than a central call per operation.”
- “The deep dives are the atomic generation switch with a propagation bound, and the cache key plus freshness rule for sign, delete-key, and revoke.”
- “I will close with the fail policy during a control-plane outage, delegation limits, and where I would change the design under stronger requirements.”

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Callers, policy model, sensitive operations, primary invariant
Estimates and contract	11:00–17:00	Decision rate, generation size, propagation bound; API semantics
Architecture	17:00–30:00	Diagram, control-plane/data-plane split, one decision trace
Deep dive	30:00–45:00	Compile and atomic switch; cache key and freshness rule
Operations and tradeoffs	45:00–50:00	Failure table, fail policy, delegation limit, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Who asks for a decision?	Certificates API, Vault/KMS API, gateways as trusted services; never end clients	Decision API is internal; the business API owns the caller-facing status
What is protected?	CAs, certificates, bundles, vaults, keys, and the policies themselves	Policy authoring is an authorized action, so delegation limits are in scope
Policy model?	OCI-style: group, verb, resource family, compartment scope, optional conditions; allow-only	Default deny with monotonic composition; no deny precedence in v1
How fast must a removed binding stop working?	Healthy routine propagation 60 s; sensitive decisions require current authority	Immediate effect revocation additionally requires signer-side ordering
Must key operations survive a policy-service outage?	Yes for reads and routine use; sensitive operations may fail closed	Agents cache a signed generation; fail policy is per action class
Where do subject attributes come from?	Identity service: groups and dynamic-group matches, versioned	Attributes are resolved server-side and versioned into the cache key

- Functional scope
 - Author policies as immutable versions with a validated action and resource vocabulary.
 - Bind a subject to a policy version at a scope
 - Delegated administrators bind only within their own authority.
 - Compile a tenant's bindings into a signed, immutable generation and switch to it atomically.
 - Evaluate `allow / deny` for (subject, action, resource) on trusted attributes.
 - Distribute generations to enforcement agents under a bounded propagation delay.
 - Record every sensitive decision with the generation that produced it.

Requirement	Target	Enforced by
Agent-local decision latency	p99 < 2 ms	In-process evaluation over a compiled index; no network call on the hot path
Central decision latency	p99 < 20 ms	Versioned attribute caches; single-region hop

Requirement	Target	Enforced by
Decision availability for routine use	99.99%	Signed generation cached on agent disk; data plane never blocks on the policy control plane
Propagation of a publication	Healthy connected agents within 60 s	Change hints, bounded polling, and exact-generation acknowledgments; partitions report partial status
Sensitive-operation freshness	0 accepted stale policy snapshots at authoritative decision time	Current policy and attributes; effect-side barrier for immediate revocation; unavailable authority means no effect
Maximum staleness for routine operations	24 h on cached generation; alarm at 1 h	Agent fail policy with signed-bundle age check
Publication latency	p99 < 5 s for 10,000 bindings	Snapshot compile, one HSM signature, one compare-and-set switch
Audit durability	100% of sensitive decisions; 7 y retention	Append-only decision log through a transactional outbox

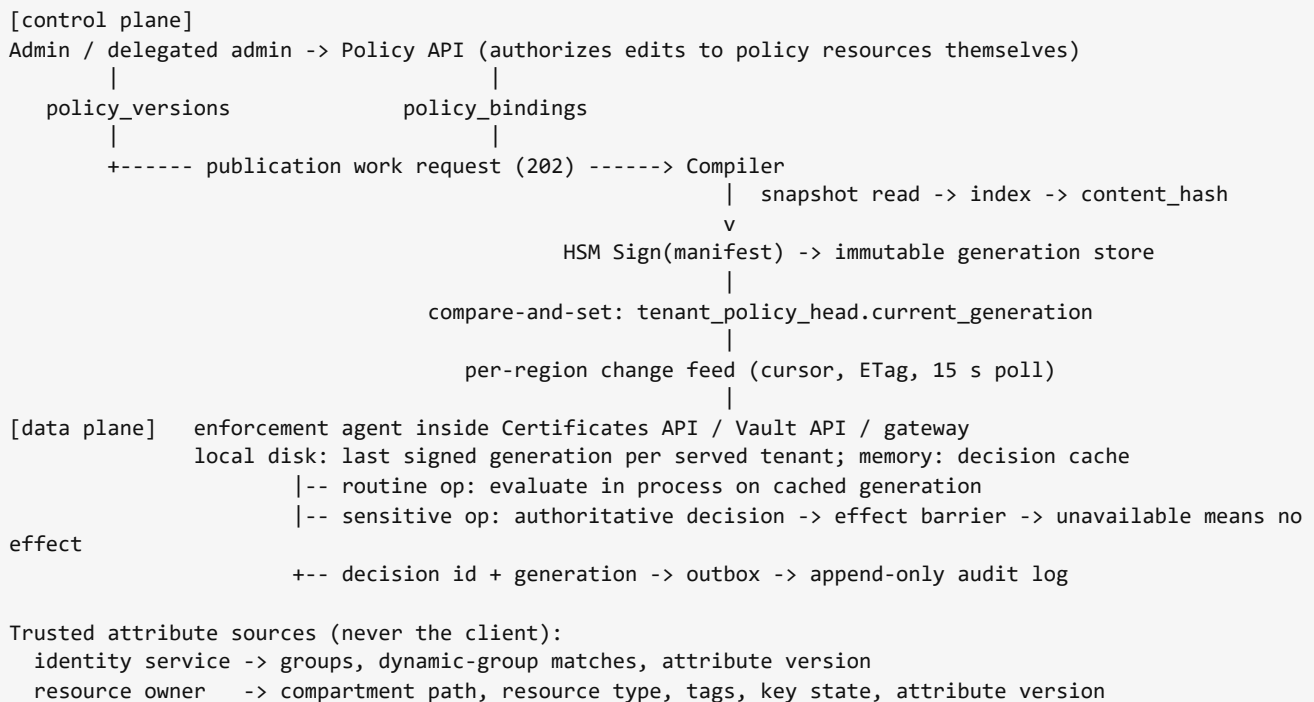
- Invariants
 - Default deny: no binding at the resource's scope means no access, for every action.
 - Policy versions and published generations are immutable
 - Changes create new ones.
 - Every decision names the generation and attribute versions it used
 - Two agents on the same inputs return the same answer.
 - Subject and resource attributes are resolved server-side from trusted sources
 - A client-supplied attribute never enters evaluation.
 - A delegated administrator cannot bind more authority than it holds at the current generation.
 - The generation signing key and the customer CA and vault keys stay inside the HSM
 - This service holds key references, never key bytes.
- Non-goals
 - Authentication and token issuance
 - Identity is an upstream dependency.
 - Cross-tenancy sharing statements in v1.
 - Request-time conditions beyond a fixed attribute vocabulary.

Capacity and consequence — IC3 baseline

- Tenants and bindings, assumptions
 - 10,000 tenants
 - Median 50 bindings, p99 5,000, hard cap 50,000 per tenant.
 - Compiled entry about 200 B: median generation 10 KB, p99 1 MB, maximum 10 MB.
 - Decision: compile per tenant, never a global bundle, so an agent fetches only the tenants it serves and a hot tenant's 10 MB never fans out fleet-wide.
 - Decision: reject over-cap publications with 422 so compile time and bundle size stay bounded.
- Decision rate, order of magnitude
 - Assume 50,000 total decisions/s across a broader platform deployment.
 - This is a separate scale scenario from s2.04's 5,500 cryptographic operations/s.
 - Label the sensitive fraction explicitly instead of assuming key operations are mostly routine.
 - With an assumed mean decision latency of 20 ms, 50,000/s requires about 1,000 concurrent evaluations.
 - A p99 latency target is not a mean-service-time measurement.
 - Decision: routine evaluation remains local.
 - Sensitive decisions use the authoritative path and must be included in its capacity and availability budget.
 - If 1% are sensitive, the authority handles 500 decisions/s.
 - If this service fronts mostly sign/decrypt calls, size it for nearly all 50,000/s instead.
- Decision cache per agent

- 100,000 entries at about 250 B each is about 25 MB.
- Decision: bound by LRU size and 60 s expiry
 - The generation in the key makes a switch a natural invalidation without a flush.
- Propagation
 - 10,000 tenants polled every 15 s by 1,000 agents is about 670,000 conditional requests/s, which is unacceptable.
 - Decision: agents poll one per-region change feed with a cursor and fetch only changed tenants
 - Steady state is about 70 requests/s per fleet.
- Publication
 - 50,000 bindings at about 10 µs each compile in about 0.5 s, plus one HSM signature at 10–50 ms.
 - Decision: publication is a work request returning 202 with an operation id, not a synchronous call.

Architecture and flow — IC3 baseline



- Policy API and metadata store own authoring.
 - Mechanism bought: immutable versions and bindings with optimistic versions, so every change is attributable and reversible.
 - Cost paid: a 99.9% control-plane path that cannot sit on the decision hot path.
- Compiler and generation store own the published truth.
 - Mechanism bought: one signed, content-addressed artifact per tenant that agents verify at rest and after a restart.
 - Cost paid: publication is asynchronous, and an orphan generation from a crashed publish must be tolerated and garbage-collected.
- Tenant head pointer owns the atomic switch.
 - Mechanism bought: one compare-and-set decides which generation is current, so two publishers cannot both win.
 - Cost paid: publishers carry `expectedGeneration` and handle 409.
- Enforcement agents own decisions.
 - Mechanism bought: routine reads can use a verified cached generation within their explicit stale window.
 - Sensitive effects pay for authoritative freshness.
 - Cost paid: staleness becomes a designed property with an explicit fail policy per action class

- Agents verify signatures and track generation age.
 - Attribute sources own subject and resource facts.
 - Mechanism bought: versioned attributes, so a group removal invalidates cached decisions without a policy change.
 - Cost paid: a second dependency with its own staleness budget.
- Vault API authenticates a resource-principal token and derives `tenant_id` and `subject_ref`; nothing in the body is treated as an attribute.
- The agent resolves subject attributes from the identity cache (groups, version `s_v`) and resource attributes from the local key record (compartment path, key state, version `r_v`).
- The agent loads the tenant's current generation `g` from memory; its signature was verified once at load.
- Cache lookup on `(tenant, subject, action, resource, g, s_v, r_v)`.
- Match trusted ancestor IDs and exact action grants.
- Evaluate supported conditions over trusted attributes.
- ⇒ No applicable grant means deny.
- For sign, obtain a current authoritative decision over policy and attribute versions.
- Bind authorization to the effect and enforce the declared signer freshness barrier.
- ⇒ If authority is unavailable, return retryable unavailable without signing.
- Allowed: the HSM performs the signature; decision id and `g` go to the audit outbox inside the operation's transaction.
- Denied: the business API returns `404` when the subject cannot inspect the resource's compartment and `403` when it can inspect but not act.

API and data model — IC3 baseline

Operation	Request	Result	Contract
POST /authorization-policies	{name, rules}	201 policy	I; privileged authoring; 422 on unknown verb or resource family
POST /authorization-policies/{id}/versions	{rules}	201 {version, contentHash}	I; compile-checked before it can be bound; immutable
PUT /policy-bindings/{id}	{subjectRef, policyVersionRef, scope}	200 binding	V; 403 when the grant exceeds the caller's effective authority; cross-tenant subjectRef is 404
DELETE /policy-bindings/{id}	—	202 publication	V; effective under the documented propagation bound
POST /policy-publications	{expectedGeneration}	202 publication	I; atomic generation switch; 409 when expectedGeneration is not current
GET /policy-publications/{id}	—	{state, generation, ackedAgentsPct}	Work-request pattern; SUCCEEDED at switch, PROPAGATED when acks pass the bound
GET /policy-generations/{generation}	—	Signed compiled bundle	Enforcement agents only; immutable; strong ETag equals contentHash
GET /policy-changes?cursor=	—	{items: [{tenantId, generation}], nextCursor}	Agent change feed; cursor bound to region

Operation	Request	Result	Contract
Internal POST /authorization-decisions	{subjectRef, action, resourceRef, context}	200 {allowed, decisionId, generation, reasonCode}	Trusted service identity; attributes resolved server-side; 200 {allowed:false} is a successful evaluation

```

CREATE TABLE policy_versions (
  tenant_id text NOT NULL, policy_id text NOT NULL, policy_version bigint NOT NULL,
  rules jsonb NOT NULL, compiled_ref text NOT NULL, content_hash text NOT NULL,
  PRIMARY KEY (tenant_id, policy_id, policy_version)
);
CREATE TABLE policy_bindings (
  tenant_id text NOT NULL, binding_id text NOT NULL, subject_ref text NOT NULL,
  resource_scope text NOT NULL, policy_id text NOT NULL, policy_version bigint NOT NULL,
  granted_by text NOT NULL, granted_at_generation bigint NOT NULL, version bigint NOT NULL,
  PRIMARY KEY (tenant_id, binding_id),
  FOREIGN KEY (tenant_id, policy_id, policy_version) REFERENCES policy_versions
);
CREATE INDEX policy_subject ON policy_bindings (tenant_id, subject_ref, resource_scope);
CREATE TABLE policy_generations (
  tenant_id text NOT NULL, generation bigint NOT NULL, manifest_ref text NOT NULL,
  content_hash text NOT NULL, signing_key_id text NOT NULL, published_at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, generation)
);
CREATE TABLE tenant_policy_head (
  tenant_id text PRIMARY KEY, current_generation bigint NOT NULL, updated_at timestamptz NOT NULL,
  FOREIGN KEY (tenant_id, current_generation) REFERENCES policy_generations
);

```

State	Key	Fields/invariant
Policy version	(tenant_id, policy_id, policy_version)	Rules, compiled reference, content hash; immutable
Binding	(tenant_id, binding_id)	Subject, scope, policy version, grantor, optimistic version; grant within grantor's authority
Generation	(tenant_id, generation)	Manifest reference, content hash, signing key id; immutable and content-addressed
Head pointer	tenant_id	current_generation; changed only by compare-and-set on the previous value
Decision audit	(tenant_id, decision_id)	Subject, action, resource, generation, attribute versions, outcome; append-only

- Conventions
 - I and v follow the [shared conventions](#).
 - Decision cache key: (tenant, subject, action, resource, policyGeneration, subjectAttributeVersion, resourceAttributeVersion) with bounded expiry.
 - Precedence and scope inheritance are defined by the policy language, never by ad hoc caller code.
- Documented OCI behavior, kept separate from the proposed design
 - Policy statements read Allow group X to manage certificate-authority-family in compartment Y, optionally with where conditions
 - See [OCI IAM policies](#) and [policy syntax](#).
 - Verbs form an inclusion ladder inspect < read < use < manage.
 - dynamic-group subjects are resources matched by rule
 - Instance and resource principals build on them
 - See [dynamic groups](#).
 - A policy attached to a compartment applies to it and its sub-compartments
 - Statements only grant, and no matching statement means no access.

- Certificates and Vault expose resource families such as `certificate-authority-family` and `keys / vaults`
- See [OCI Certificates](#) and [OCI Vault](#).
- The 60 s and 5 s freshness targets in this packet are proposed interview requirements.
- They are not quoted OCI IAM propagation guarantees.
- Proposed design: immutable versions, signed generations, agents, cache keys, and the freshness rule are this answer's proposal, not a description of how OCI IAM is built.

Compile, sign, and atomic generation switch — IC3 baseline

- Consistent input
 - Compile one tenant snapshot with an explicit source revision.
 - Include bindings, policy versions, and the supported condition vocabulary.
 - A later mutation schedules another publication through its transactional outbox.
- Compiled structure
 - Map trusted subject identities to scoped action grants and conditions.
 - Use immutable compartment ancestor IDs or delimiter-aware path components.
 - A plain string prefix would incorrectly treat `/a` as an ancestor of `/ab`.
 - Expand convenience verbs into exact endpoint actions before evaluation.
- Signed manifest
 - Sign the canonical tuple containing purpose, tenant, generation, parent generation, source revision, schema, and content hash.
 - Signing only content bytes does not bind unsigned tenant or generation metadata.
 - Agents pin verification keys independently and validate the entire manifest before applying it.
- Publish flow
 - Store content by hash before activating it.
 - Bind the winning head to one exact signed manifest.

```
PUBLISH(tenant, expectedGeneration):
    snapshot, sourceRevision = consistent_snapshot(tenant)
    bundle = compile_and_validate(snapshot)
    manifest = canonical(purpose, tenant, expectedGeneration + 1,
                        expectedGeneration, sourceRevision, schema, hash(bundle))
    signature = sign(manifest)
    put_immutable(hash(bundle), bundle)
    put_immutable(hash(manifest, signature), manifest, signature)
    verify stored bytes by hash
    transaction:
        require tenant head == expectedGeneration
        require source revision still matches, or explicitly schedule the newer revision
        advance head to exact manifest hash and new generation
        persist generation record, audit, and change outbox
    return committed publication; track propagation separately
```

- Concurrent publishers
 - Only one expected-head transaction wins.
 - Different content candidates do not overwrite one shared tenant/generation object.
 - Unreferenced candidates can be removed after the retention safety window.
- Propagation evidence
 - Acknowledgments name the exact generation and manifest hash.
 - A 60 s timer expiry reports delayed or partial propagation, not success.
 - Require every mandatory serving cohort or an explicitly approved exclusion before claiming full propagation.
- Rollback
 - Publish a new higher generation containing previously reviewed rules.
 - Re-evaluate those rules against current identities and resource attributes.

- An old policy may reintroduce access that was intentionally removed.
- Do not lower the head and bypass agent anti-rollback checks.

Decision evaluation, caching, and freshness — IC3 baseline

- Trusted inputs
 - Resolve principal and group membership from the identity authority.
 - Resolve resource ownership and ancestor IDs from the owning service.
 - Map the endpoint to an exact action.
 - Derive request context from authenticated server observations.
 - Do not trust a groups, tenant, or compartment field from the request body.
- Evaluation
 - Default deny when no applicable grant permits the action.
 - Match tenant and scope before evaluating action and conditions.
 - An allow-only language still permits access removal through binding deletion or changed conditions.
 - Policy diffs are not generally monotonic increases in access.
- Cache identity
 - Use tenant, subject, action, resource, policy generation, subject version, and resource version.
 - A versioned cache key only helps after the evaluator learns the newer version.
 - Bound freshness of policy and both attribute sources separately.
- Normal propagation target
 - Assume connected routine evaluators converge within 60 s during healthy operation.
 - This is a propagation objective, not an unconditional revocation guarantee during partitions.
- Routine outage policy
 - Explicitly low-risk bundle reads may use signed policy and trusted attributes up to 24 h stale.
 - Record that decision as stale and alarm at 1 h.
 - This exception means routine access revocation can take longer than 60 s during an outage.
- Sensitive actions
 - Include signing and decrypting customer material.
 - Include key lifecycle changes and certificate revocation.
 - Include policy or binding changes.
 - Baseline requires authoritative current policy and attributes for these decisions.
 - If the authority cannot answer, return a retryable unavailable result and perform no effect.
 - Do not fabricate empty groups and report a definitive policy denial.
- Decision-to-effect race
 - An authoritative decision can become stale before the effect executes.
 - Bind the decision to action, resource version, and a short single-use grant.
 - Immediate revocation requires the effect authority to order grant consumption against revocation or enforce an equivalent barrier.
 - A 5 s or 60 s lease instead promises a bounded stale-use window.
- Delegated administration
 - Check administrative scope and exact granted actions under current policy.
 - Prove that granted conditions are no broader than the delegator's authority in the supported finite policy language.
 - Reject grants whose condition implication cannot be established.
 - A simple bitset subset check is insufficient for arbitrary conditions.
 - State whether grants persist after the grantor loses authority and audit orphaned grants.
- Existence hiding
 - Without inspect permission, return 404 for inaccessible resource IDs.
 - With inspect but without the requested action, return 403.

- Infrastructure unavailability remains a separate retryable outcome.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Policy control plane down	Authoring and publication, all tenants; no decision impact	Health checks; publication backlog	Agents decide on cached generations; sensitive ops fail closed after 60 s unconfirmed	Fail over stateless fleet; agents resume the feed from cursor
Metadata store unavailable	New versions, bindings, head switches	Write error rate	Decisions unaffected; publications return 503	Fail over replica; retry with the same idempotency key
Identity service down	Attribute refresh for all agents	Attribute cache age	Routine ops on cached attributes to 24 h; sensitive to 10 min, then deny	Restore; cache keys change via s_v
HSM publication key unavailable	New policy publications	Signing errors	Existing routine policies remain usable within validity; sensitive evaluation uses authoritative state	Recover signing and publish pending changes; no fabricated propagation success
Agent stuck on old generation	One agent's tenants	Heartbeat lag beyond 60 s	Publication reports laggards; sensitive ops there go live or deny	Drain the agent; re-fetch from feed
Over-permissive compile bug	One tenant per generation; all tenants if the compiler is wrong	Authoring-time simulation diff; canary tenants	Pointer rolled back to previous generation	Compare-and-set to prior generation; fix; re-publish
Concurrent publishes	One tenant	409 on head switch	Loser retries on the new head	None needed
Attribute injection attempt	One request	Schema rejects body attributes; audit reason code	Evaluated on server-resolved attributes only	Alert on repeated attempts per principal
Audit outbox backlog	Sensitive decision records	Outbox lag	Operations continue; records delayed, not lost	Relay catches up; alarm at 5 min lag

- Observe
 - Decision and deny rate per tenant and action.
 - Generation age across agents
 - Oldest unconfirmed generation serving sensitive actions.
 - Publication latency and time to PROPAGATED.
 - Central evaluator call rate, which rises when agents lose freshness.
 - Orphaned grants and over-cap tenants.
- Retry discipline
 - Publications retry with the same idempotency key and expectedGeneration
 - A 409 means re-read the head, not retry blindly.
 - Agents back off with jitter on the change feed and keep serving the last verified generation.
 - A deny is terminal for the caller
 - A freshness 503 is retryable with bounded attempts.
- Tenant isolation
 - Generations are per tenant and fetched with a tenant-scoped grant
 - An agent never receives another tenant's bindings.
 - Binding subjects and scopes must belong to the caller's tenancy
 - Cross-tenant references return 404.
 - Quotas cap bindings, versions, and publications per tenant so one tenant cannot starve the compiler.
 - HSM-held keys remain the custody boundary
 - Authorization decides whether a signing call may happen, never where key bytes live.

- Rollout and rollback safety
 - Vocabulary changes are additive
 - An agent that does not know a verb denies it, so new agents roll out before policies use the verb.
 - Bundle formats carry a version
 - Agents accept the current and previous format.
 - Canary the compiler on internal tenants and diff decisions against the previous generation before fleet rollout.
 - Rolling back an agent version must not roll back its cached generation.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Central decision per operation	Authoritative evaluation	Current decision inputs	Extra dependency; decision-to-effect race remains	Sensitive actions with explicit effect-side freshness enforcement
Signed generation in agents	Compiled per-tenant bundle, local evaluation	Static stability; deterministic decisions	Staleness must be designed; agents verify and track age	Baseline for key and certificate data planes
Role table per service	Local roles in each service	Simple	Divergent semantics; no generation; cache by role alone	Single-service prototypes
Capability tokens	Caller presents a signed grant per resource	No lookup at enforcement	Revocation is hard; token sprawl	Short-lived scoped access such as one signing session
Generic config distribution	Reuse a signed config channel	One distribution system	No decision semantics, cache key, or freshness rule	A signed config channel exists and the language is tiny

- IC4 stretch — stronger revocation guarantee
 - Requirement: a removed binding must not sign anywhere after 5 s.
 - Change: consume a scoped authorization lease at the signing boundary before its 5 s deadline.
 - Bound policy and attribute freshness together, including clock allowance.
 - This is a five-second stale-use contract, not immediate revocation.
 - Cost: the central evaluator becomes a hot dependency for sensitive actions and must be sized and celled like a data plane.
- IC4 stretch — explicit deny
 - Add deny statements with documented precedence, compiled into the same bitset form.
 - Cost: deny precedence complicates reasoning further.
 - Even allow-only binding removals already require access-loss simulation.
- IC4 stretch — 10× load
 - Routine evaluation remains agent-local.
 - Sensitive traffic requires an authoritative service sized for its actual fraction of total demand.
 - Shard the compiler by tenant hash
 - Large tenants compile incrementally from the parent generation.
 - A 1% sensitive fraction at 500,000 decisions/s produces 5,000 authoritative decisions/s.
 - Read replicas must meet the required freshness barrier
 - An arbitrary stale replica is insufficient.
- IC4 stretch — cost and blast radius
 - Cell the policy control plane per region with tenant-to-cell mapping so a compiler or store failure strands one cell's publications, never all.
 - Bundle storage is cheap
 - The cost center is central-evaluator capacity for freshness checks, which the sensitive-action list directly controls.

- Weakest assumption
 - The weakest assumption is that sensitive operations can afford authoritative evaluation and effect-side freshness enforcement.
 - Mitigation: keep `sign` in the sensitive set, keep that set small, and measure the real bound with per-agent generation heartbeats.
- Over-engineering warning
 - Do not build a general attribute-based language with request-time expressions in v1
 - A fixed verb ladder over compartment prefixes is enough.
 - Do not centralize every decision for freshness
 - That trades data-plane availability for a guarantee only sensitive actions need.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Trusted callers, allow-only model, sensitive action list, 60 s bound	Negotiates the bound per action class and prices a 5 s bound	Treats freshness as a requirement with a cost, not a TTL
Estimates	Decision rate forces agent-local evaluation; bundle size forces per-tenant compile	Change-feed fan-out math; evaluator sized from the sensitive fraction	A number moves evaluation out of the central service
Architecture	Authoring plane, signed generations, agents, attribute sources	Cost of each split; region cells; signing-key rotation with overlap	Static stability stated together with its fail policy
Deep dive	Signed scope, atomic head and outbox, versioned cache	Conditional delegation and decision-to-effect ordering	Explains both stale inputs and post-decision races
Operations	Explicit stale-read exception and sensitive failure policy	Higher-generation rollback and cohort evidence	A timer never proves propagation
Wrap	Recap and the caching-versus-freshness tradeoff	Names the 60 s window as the weakest assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Why not cache decisions by role, like most services do?
 - **IC3 baseline**
 - A role-only key misses tenant, resource, and policy version, so a policy change or compartment move keeps serving the old answer.
 - **IC4 stretch**
 - The key carries generation and attribute versions, so invalidation is structural and never needs a fleet-wide flush.
- Q2: The client already knows its groups
 - Why not let it send them?
 - **IC3 baseline**
 - Any client-supplied trusted attribute is a privilege-escalation path
 - The agent resolves attributes from the identity service only.
 - **IC4 stretch**
 - Bind the attribute version into the decision id so an audit proves which membership snapshot allowed a signature.
- Q3: An operator is removed
 - How long can they still sign?
 - **IC3 baseline**
 - Sensitive signing requires current policy and attributes.
 - Immediate effect revocation additionally needs an ordered signer barrier.

- Routine outage exceptions can retain read access for up to 24 h.
- **IC4 stretch**
 - Lease-based push at 5 s for sensitive actions, with the central evaluator sized as a data-plane dependency.
- Q4: The policy service is down for six hours
 - What happens to TLS and key operations?
 - **IC3 baseline**
 - Agents keep deciding on the last signed generation
 - Reads and routine use continue
 - Sensitive actions require current authority and perform no effect when that authority is unavailable.
 - **IC4 stretch**
 - An emergency override is a separately authorized and audited weakening of the revocation bound.
 - Never describe that override as preserving the original security guarantee.
- Q5: A delegated admin grants themselves `manage` on the whole tenancy.
 - **IC3 baseline**
 - The binding API checks the grant is a subset of the caller's effective authority at the current generation and returns `403`.
 - **IC4 stretch**
 - Argue whether grants should cascade-revoke when the grantor loses authority, and why the baseline keeps them with an orphan report.
- Q6: Two admins publish at the same time.
 - **IC3 baseline**
 - The head switch is a compare-and-set on `expectedGeneration`
 - One wins, the other gets `409` and republishes on the new head.
 - **IC4 stretch**
 - The loser's orphan bundle is harmless because generations are immutable and content-addressed
 - A sweeper removes it.
- Q7 (premise reversal): Why build a policy service when OCI IAM already evaluates `allow group X to manage certificate-authority-family in compartment Y`?
 - **IC3 baseline**
 - If the platform's IAM covers the vocabulary and freshness, build only the enforcement agent and the resource attribute resolver
 - This design is the shape of that evaluation component, not a replacement for IAM.
 - **IC4 stretch**
 - The build case is a vocabulary or freshness rule the platform cannot express, and that gap must be named before adding a system.
- Q8: Is a signed bundle needed if agents fetch over mTLS?
 - **IC3 baseline**
 - mTLS protects the transport
 - The signature protects the artifact at rest, which an agent restarting from disk during an outage must verify.
 - **IC4 stretch**
 - The signing key rotates with overlap under its own `signing_key_id`, so rotation never invalidates cached generations.
- Validation scenarios
 - Binding removed
 - Sensitive action denied through live evaluation inside the bound.
 - Publish with a stale `expectedGeneration`.
 - Agent restart from disk with the control plane unreachable.
 - Group membership removed with no policy change.

- Request body carrying a `groups` field.
- Delegated grant exceeding the grantor's authority.
- Tenant A token on a tenant B key id.
- Compiler regression caught by decision diff on a canary tenant.

Recall card

- "Default deny over a signed, immutable per-tenant generation."
- "The cache key carries the generation and the attribute versions, so invalidation is structural."
- "Sensitive actions need a confirmed-fresh generation or a live answer; unreachable means deny."
- "Routine cached reads survive within a declared stale window; sensitive effects need fresh authority."
- "A delegate grants only a subset of what it holds, and cross-tenant ids are `404`."
- Wrap-up script
 - "The design compiles immutable policy versions and bindings into a signed per-tenant generation, switched atomically and cached by enforcement agents in the data plane."
 - "The key invariant is default deny on server-resolved attributes, with every decision tied to the generation and attribute versions that produced it."
 - "The main tradeoff is cached stability versus freshness; I resolve it per action class, so signing and deletion need a confirmed-fresh generation while reads survive a control-plane outage."
 - "My weakest assumption is that a `60 s` propagation bound is acceptable for revoking a signing right; I would measure the real bound with agent heartbeats first."

References

- [Ranked question and provenance](#).
- [OCI role framing](#).
- [Detailed API and schema companion](#).
- Reference check: 2026-09-09.
 - Official documentation supports the OCI policy statement shape, verb ladder, dynamic groups, and compartment inheritance cited above.
 - Capacity values, the `60 s` bound, and the generation architecture remain proposed interview assumptions.
- [OCI IAM policies](#): policy attachment to compartments and inheritance down the tree.
- [OCI policy syntax](#): Allow `<subject>` to `<verb>` `<resource-type>` in `<scope>` where `<conditions>`
 - Verb ladder.
- [OCI dynamic groups](#): resources as policy subjects
 - Basis for instance and resource principals.
- [OCI Certificates overview](#): IAM gating of CA and certificate operations.
- [OCI Vault key overview](#): HSM-held keys
 - The operations this service authorizes.
- [PKI and KMS mechanisms](#): Part B.4 authorization sequence, B.5 OCI equivalents, key custody and `Sign` semantics.
- [Multi-tenant certificate authority](#): the sibling data plane this service gates
 - `404`-not-`403` isolation test.